Expression reference (XMILE / l3fp / Lua)

XMILE shows XMILE v1.0 syntax, **l3fp** shows the `\fp_eval`-compatible form used inside `\mrule`, and **Lua** shows the equivalent Lua expression (Lua 5.3, the version embedded in LuaTeX). Orange entries are not yet fully supported by numodel's rule renderer (the expression computes correctly, but the keyword in the rule table is not yet translated). *Italic* cells contain a derived equivalent rather than a native keyword.

Table 1: Expression reference: XMILE, `\fpeval`, and Lua

XMILE	l3fp (`\fpeval`)	Lua (5.3)
Arithmetic operators		
+	+	+
-	-	-
*	*	*
/	/	/
^	^	^
MOD(x,y)	*x - trunc(x/y)*y*	x % y
Comparison operators		
<	<	<
<=	<=	<=
>	>	>
>=	>=	>=
=	=	==
<>	!=	~=
Boolean operators		
AND	&&	and
OR	||	or
NOT	!	not
Control flow		
IF c THEN a ELSE b	c ? a : b	*c and a or b*
General math functions		
ABS(x)	abs(x)	math.abs(x)
SIGN(x)	sign(x)	*x>0 and 1 or (x<0 and -1 or 0)*
SQRT(x)	sqrt(x)	math.sqrt(x)
INT(x)	trunc(x)	*(math.modf(x))*
INT(x+0.5)	round(x)	*math.floor(x+0.5)*
INT(x)	floor(x)	math.floor(x)
-INT(-x)	ceil(x)	math.ceil(x)
MIN(x,y)	min(x,y)	math.min(x,y)
MAX(x,y)	max(x,y)	math.max(x,y)
—	fact(x)	—
INT(LOG10(ABS(x)))	logb(x)	*math.floor(math.log(math.abs(x),10))*
Exponential and logarithmic		
EXP(x)	exp(x)	math.exp(x)
LN(x)	ln(x)	math.log(x)
LOG10(x)	*ln(x)/ln(10)*	math.log(x,10)
Trigonometry (radians)		
SIN(x)	sin(x)	math.sin(x)
COS(x)	cos(x)	math.cos(x)
TAN(x)	tan(x)	math.tan(x)
ARCSIN(x)	asin(x)	math.asin(x)
ARCCOS(x)	acos(x)	math.acos(x)

Continued on next page

Table 1: Expression reference: XMILE, `\fpeval`, and Lua (Continued)

XMILE	l3fp (`\fpeval`)	Lua (5.3)
`ARCTAN(x)`	`atan(x)`	`math.atan(x)`
`ARCTAN2(y,x)`	`atan(y,x)`	`math.atan(y,x)`
1/TAN(x)	`cot(x)`	*1/math.tan(x)*
1/SIN(x)	`csc(x)`	*1/math.sin(x)*
1/COS(x)	`sec(x)`	*1/math.cos(x)*
ARCSIN(1/x)	`acsc(x)`	*math.asin(1/x)*
ARCCOS(1/x)	`asec(x)`	*math.acos(1/x)*
ARCTAN(1/x)	`acot(x)`	*math.atan(1/x)*
ARCTAN2(x,y)	`acot(y,x)`	*math.atan(x,y)*
Trigonometry (degrees)		
*SIN(PI/180*x)*	`sind(x)`	*math.sin(math.rad(x))*
*COS(PI/180*x)*	`cosd(x)`	*math.cos(math.rad(x))*
*TAN(PI/180*x)*	`tand(x)`	*math.tan(math.rad(x))*
*180/PI*ARCSIN(x)*	`asind(x)`	*math.deg(math.asin(x))*
*180/PI*ARCCOS(x)*	`acosd(x)`	*math.deg(math.acos(x))*
*180/PI*ARCTAN(x)*	`atand(x)`	*math.deg(math.atan(x))*
*180/PI*ARCTAN2(y,x)*	`atand(y,x)`	*math.deg(math.atan(y,x))*
*1/TAN(PI/180*x)*	`cotd(x)`	*1/math.tan(math.rad(x))*
*1/SIN(PI/180*x)*	`cscd(x)`	*1/math.sin(math.rad(x))*
*1/COS(PI/180*x)*	`secd(x)`	*1/math.cos(math.rad(x))*
*180/PI*ARCSIN(1/x)*	`acscd(x)`	*math.deg(math.asin(1/x))*
*180/PI*ARCCOS(1/x)*	`asecd(x)`	*math.deg(math.acos(1/x))*
*180/PI*ARCTAN(1/x)*	`acotd(x)`	*math.deg(math.atan(1/x))*
*180/PI*ARCTAN2(x,y)*	`acotd(y,x)`	*math.deg(math.atan(x,y))*
Constants		
`PI`	`pi`	`math.pi`
`e`	*exp(1)*	*math.exp(1)*
`INF`	`inf`	`math.huge`
`NAN`	`nan`	*0/0*
PI/180	`deg`	*math.pi/180*
1	`true`	`true`
0	`false`	`false`
Simulation-specific functions (XMILE only)		
`TIME`	*user variable (`\mvar`)*	*user variable*
`DT`	*user variable (`\mvar`)*	*user variable*
`STEP(h,t0)`	*T >= t0 ? h : 0*	*(T >= t0) and h or 0*
`RAMP(s,t0)`	*T > t0 ? s*(T-t0) : 0*	*(T > t0) and s*(T-t0) or 0*
`DELAY(x,dt)`	*not supported*	*not supported*
`SMOOTH(x,t)`	*not supported*	*not supported*
`INIT(x)`	*not supported*	*not supported*
`PREVIOUS(x)`	*not supported*	*not supported*
RANDOM(0,1)	`rand()`	`math.random()`
`RANDOM(lo,hi)`	*lo + (hi-lo)*rand()*	*lo + (hi-lo)*math.random()*
not supported	`randint(n)`	`math.random(n)`
not supported	`randint(m,n)`	`math.random(m,n)`

Flattening `math` into the rule environment

The verbose `math.` prefix in front of every call is the main cost of using raw Lua expressions in user-authored rules. Lua lets you eliminate it by handing the loaded expression its own environment. The pattern (Lua 5.3,

the version embedded in LuaTeX):

```
local function make_env(vars)
  local env = {}
  for k, v in pairs(math) do env[k] = v end  -- sin, cos, pi, huge, ...
  setmetatable(env, { __index = vars })      -- model variables fall through
  return env
end

local function compile_rule(expr, env)
  local f, err = load("return " .. expr, "rule", "t", env)
  if not f then error(err) end
  return f
end
```

The Euler integrator then evaluates each rule with no boilerplate on the user's side:

```
local env = make_env(vars)
local f   = compile_rule("v + g * dt", env)  -- user-typed expression
vars.v    = f()                              -- vars.g, vars.dt via __index
```

Three things to note. First, copying `math` into `env` once gives a direct table lookup for `sin`, `cos`, …; only model-variable reads pay the metatable hop, and only on first access per step. Second, the `"t"` mode flag on `load` restricts the chunk to text (no precompiled bytecode), a useful sandbox guarantee for user input. Third, name collisions matter: if a model variable is called `sin`, `log`, `pi`, etc., the math binding in `env` shadows it. In practice this is a non-issue (physics models do not name a stock `sin`), but it is worth a sanity check at `register` time.

The next table repeats the expression reference under this assumption: the `math.` prefix is gone everywhere. Entries that were derived stay derived; `numodel` could also inject named helpers for `sign`, `cot`, `csc`, `round`, etc. into `env` to make those native too, but that is outside the standard library and not assumed here.

One subtlety worth flagging: in radian-mode trig, `deg` is the *function* `math.deg` (radians→degrees), so the constant "`deg`" for $\pi/180$ has to be spelled out as `pi/180`.

Expression reference (XMILE / l3fp / Lua, flattened)

Table 2: Expression reference with `math` flattened into the rule environment

XMILE	l3fp (`\fpeval`)	Lua (math flattened)
Arithmetic operators		
+	+	+
-	-	-
*	*	*
/	/	/
^	^	^
MOD(x,y)	*x - trunc(x/y)*y*	x % y
Comparison operators		
<	<	<
<=	<=	<=
>	>	>
>=	>=	>=
=	=	==
<>	!=	~=
Boolean operators		
AND	&&	and
OR	||	or

Continued on next page

Table 2: Expression reference with `math` flattened into the rule environment (Continued)

XMILE	**l3fp (\fpeval)**	**Lua (math flattened)**
`NOT`	`!`	`not`
Control flow		
`IF c THEN a ELSE b`	`c ? a : b`	*c and a or b*
General math functions		
`ABS(x)`	`abs(x)`	`abs(x)`
`SIGN(x)`	`sign(x)`	*x>0 and 1 or (x<0 and -1 or 0)*
`SQRT(x)`	`sqrt(x)`	`sqrt(x)`
`INT(x)`	`trunc(x)`	*(modf(x))*
INT(x+0.5)	`round(x)`	*floor(x+0.5)*
INT(x)	`floor(x)`	`floor(x)`
-INT(-x)	`ceil(x)`	`ceil(x)`
`MIN(x,y)`	`min(x,y)`	`min(x,y)`
`MAX(x,y)`	`max(x,y)`	`max(x,y)`
—	`fact(x)`	—
INT(LOG10(ABS(x)))	`logb(x)`	*floor(log(abs(x),10))*
Exponential and logarithmic		
`EXP(x)`	`exp(x)`	`exp(x)`
`LN(x)`	`ln(x)`	`log(x)`
`LOG10(x)`	*ln(x)/ln(10)*	`log(x,10)`
Trigonometry (radians)		
`SIN(x)`	`sin(x)`	`sin(x)`
`COS(x)`	`cos(x)`	`cos(x)`
`TAN(x)`	`tan(x)`	`tan(x)`
`ARCSIN(x)`	`asin(x)`	`asin(x)`
`ARCCOS(x)`	`acos(x)`	`acos(x)`
`ARCTAN(x)`	`atan(x)`	`atan(x)`
`ARCTAN2(y,x)`	`atan(y,x)`	`atan(y,x)`
1/TAN(x)	`cot(x)`	*1/tan(x)*
1/SIN(x)	`csc(x)`	*1/sin(x)*
1/COS(x)	`sec(x)`	*1/cos(x)*
ARCSIN(1/x)	`acsc(x)`	*asin(1/x)*
ARCCOS(1/x)	`asec(x)`	*acos(1/x)*
ARCTAN(1/x)	`acot(x)`	*atan(1/x)*
ARCTAN2(x,y)	`acot(y,x)`	*atan(x,y)*
Trigonometry (degrees)		
*SIN(PI/180*x)*	`sind(x)`	*sin(rad(x))*
*COS(PI/180*x)*	`cosd(x)`	*cos(rad(x))*
*TAN(PI/180*x)*	`tand(x)`	*tan(rad(x))*
*180/PI*ARCSIN(x)*	`asind(x)`	*deg(asin(x))*
*180/PI*ARCCOS(x)*	`acosd(x)`	*deg(acos(x))*
*180/PI*ARCTAN(x)*	`atand(x)`	*deg(atan(x))*
*180/PI*ARCTAN2(y,x)*	`atand(y,x)`	*deg(atan(y,x))*
*1/TAN(PI/180*x)*	`cotd(x)`	*1/tan(rad(x))*
*1/SIN(PI/180*x)*	`cscd(x)`	*1/sin(rad(x))*
*1/COS(PI/180*x)*	`secd(x)`	*1/cos(rad(x))*
*180/PI*ARCSIN(1/x)*	`acscd(x)`	*deg(asin(1/x))*
*180/PI*ARCCOS(1/x)*	`asecd(x)`	*deg(acos(1/x))*

Continued on next page

Table 2: Expression reference with `math` flattened into the rule environment (Continued)

XMILE	**l3fp (`\fpeval`)**	**Lua (math flattened)**
*180/PI*ARCTAN(1/x)*	`acotd(x)`	*deg(atan(1/x))*
*180/PI*ARCTAN2(x,y)*	`acotd(y,x)`	*deg(atan(x,y))*
Constants		
`PI`	`pi`	`pi`
`e`	*exp(1)*	*exp(1)*
`INF`	`inf`	`huge`
`NAN`	`nan`	*0/0*
PI/180	`deg`	*pi/180*
1	`true`	`true`
0	`false`	`false`
Simulation-specific functions (XMILE only)		
`TIME`	*user variable (`\mvar`)*	*user variable*
`DT`	*user variable (`\mvar`)*	*user variable*
`STEP(h,t0)`	*T >= t0 ? h : 0*	*(T >= t0) and h or 0*
`RAMP(s,t0)`	*T > t0 ? s*(T-t0) : 0*	*(T > t0) and s*(T-t0) or 0*
`DELAY(x,dt)`	*not supported*	*not supported*
`SMOOTH(x,t)`	*not supported*	*not supported*
`INIT(x)`	*not supported*	*not supported*
`PREVIOUS(x)`	*not supported*	*not supported*
RANDOM(0,1)	`rand()`	`random()`
`RANDOM(lo,hi)`	*lo + (hi-lo)*rand()*	*lo + (hi-lo)*random()*
not supported	`randint(n)`	`random(n)`
not supported	`randint(m,n)`	`random(m,n)`